Project 2: Building Imaging Tools with Deep Learning Report

By: Haojun Zhuang, James Dinh, Aaron Shiang Yan, Abtin Khashayar

Architecture Comparison:

CNN Classifier:

Our best CNN classifier achieved a validation accuracy of **99.5%** at the 40th epoch. The architecture is composed of 5 convolution layers, one of which (fourth convolution) enhances the expressiveness of the model, outputting the same number of output channels as the input.

The architecture consistently used batchnormalization along with maxpooling after performing our convolutions. Adaptive average pooling is used after the last convolutional layer, where we then apply dropout with a probability of 0.3 and then pass through two fully connected linear layers. We then use softmax to predict our class probabilities.

UNet Segmentation Model:

Our best UNet segmentation model achieved a final validation IoU score of **99.84%**. The architecture is composed of an encoder, to downsample the image, in addition to a decoder which builds the resolution back up while reducing the number of channels. There are 4 encoder stages (including the bottleneck) and 3 decoder stages, all of which are composed of convolution blocks. Each encoder is composed of two convolutional layers, where the second serves as increasing the expressiveness of the model (output the same number of channels as the input). However, the decoder is applied after using the ConvTranspose2d function to reduce the number of channels, but to increase the image resolution. There is also a bottleneck layer, serving as the last encoder block before transitioning to the decoding phase. Skip connections are also used in the model architecture, where we utilize the cat function of the torch module. This is where we run the decoder after concatenating the upsampled result with the corresponding encoder result.

Comparison:

Both the CNN and UNet model use convolution blocks, but one main distinction is that the UNet does not use any linear layers. Instead, the output of the UNet model after the last convolution block is passed through a sigmoid function to scale the pixel logit values, in which then we compute the IoU score between the output of the model and the actual data from our validation set. As mentioned, the UNet is composed of an encoder and decoder section, where the encoder downsamples the image to produce more channels, while also taking advantage of upsampling afterwards. The UNet also uses skip connections unlike the CNN classifier.

Performance Analysis:

1. CNN Classifier:

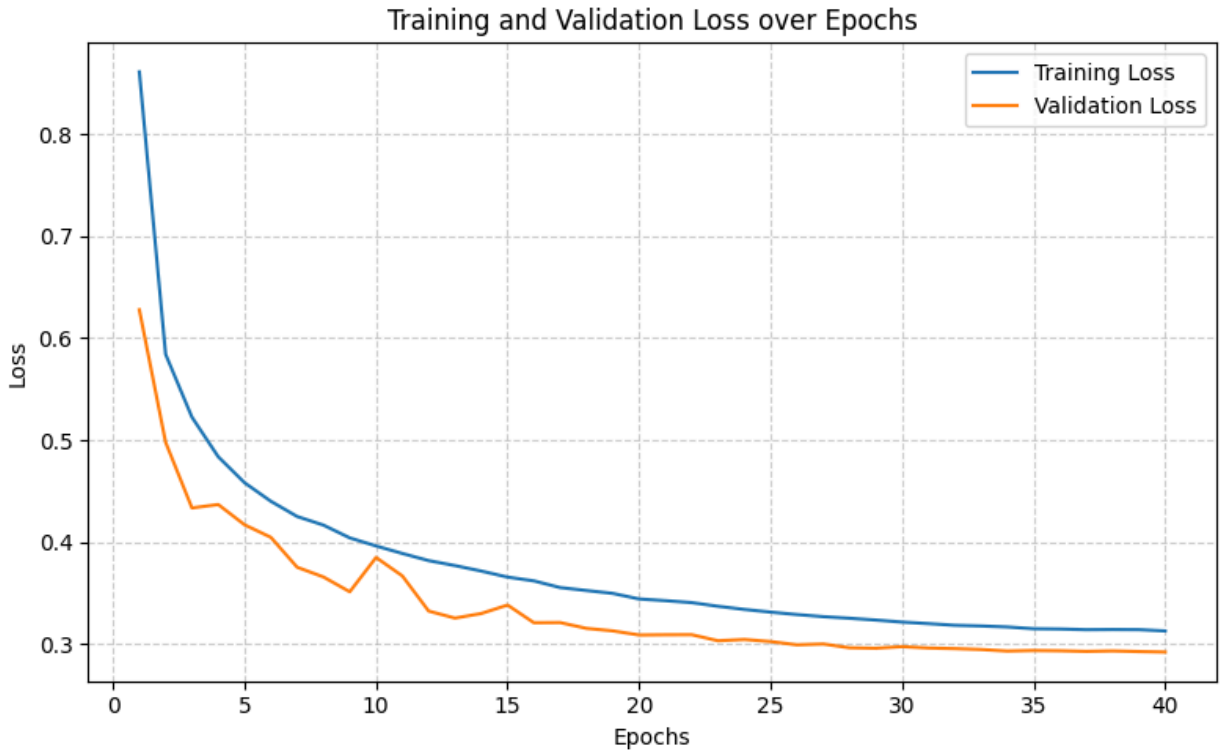
a. Best validation accuracy: 99.5%
b. Num layers: 6
c. Channels
    i. First layer outputs 64, and last convolutional layer outputs 512 channels
d. Batchnorm
e. Dropout = 0.3
f. Learning rate: .001
g. Epochs: 40
h. Weight decay: .0001
i. Batch size: 64
j. Data augmentation
k. Learning rate scheduler through CosineAnnealing
l. CrossEntropy loss with label smoothing = 0.5
m. Optimizer is AdamW
n. Max steps per epoch = None (use full dataset)

2. UNet Segmentation Model:

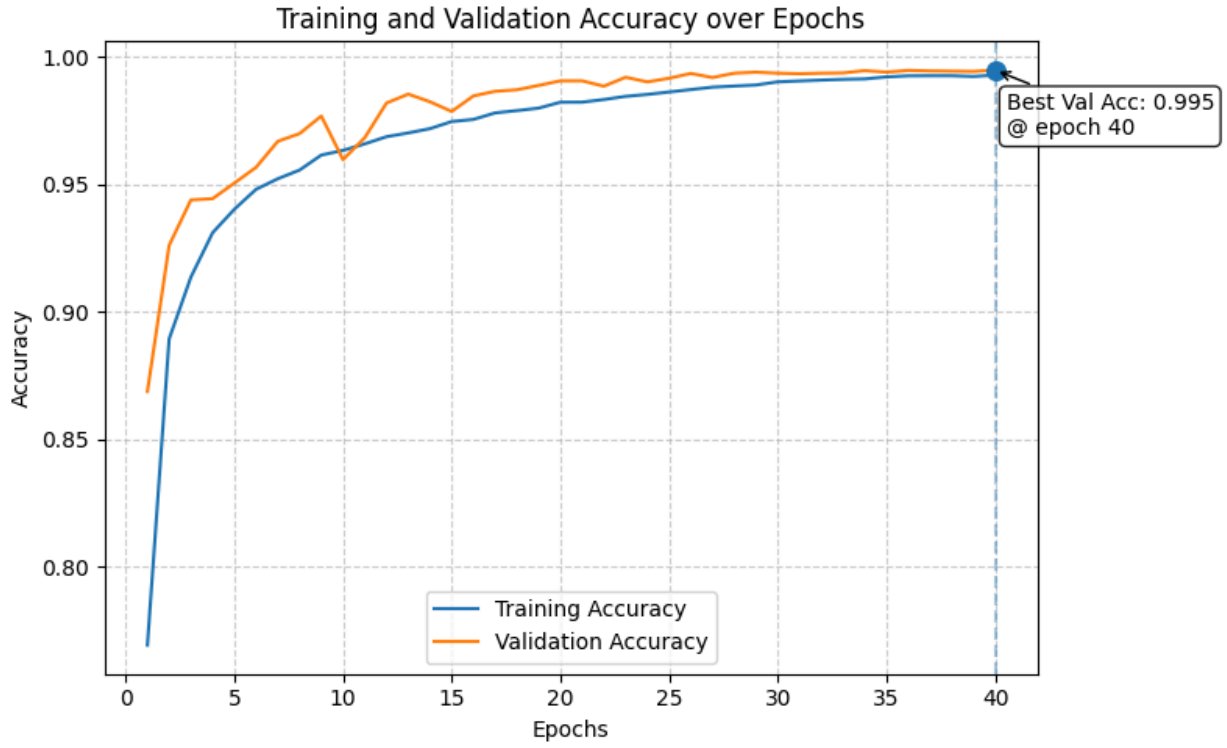
a. Validation IoU score: 99.16%
b. 4 encoder blocks (1 bottleneck) and 3 decoder blocks
c. Maximum channels = 512
d. Epochs = 1
e. Learning rate = 0.001
f. Weight decay = 0.0

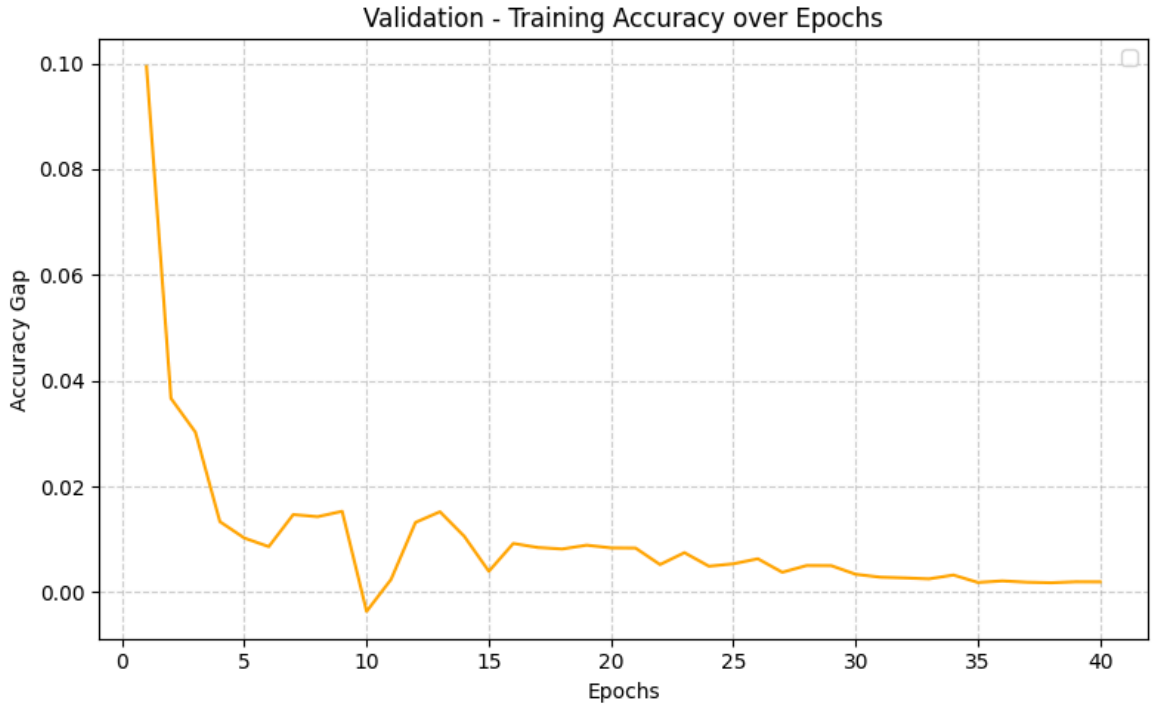
g. Optimizer: Adam
h. Skip connections
i. Batch size = 32
j. Max steps per epoch = 200

Loss Plot:



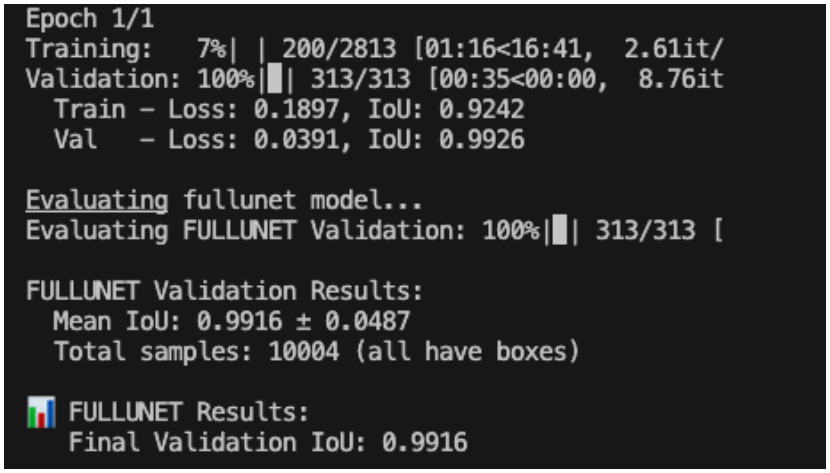
Accuracy Plot:

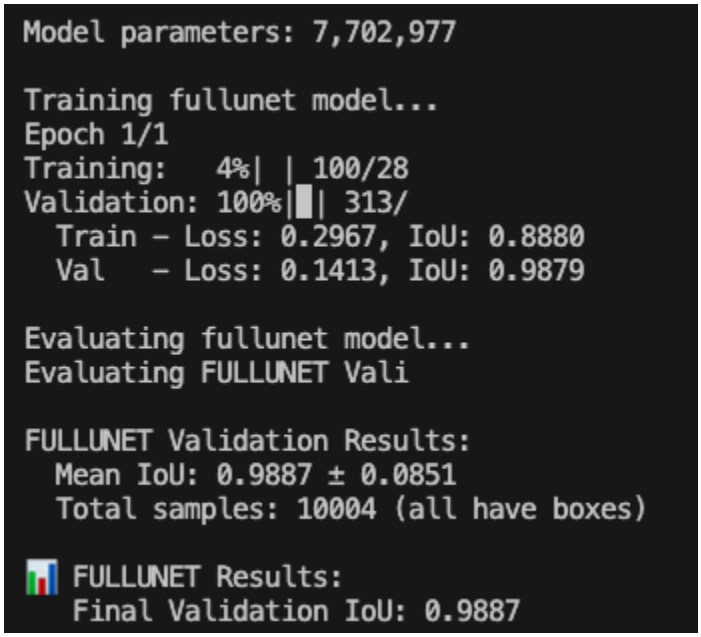




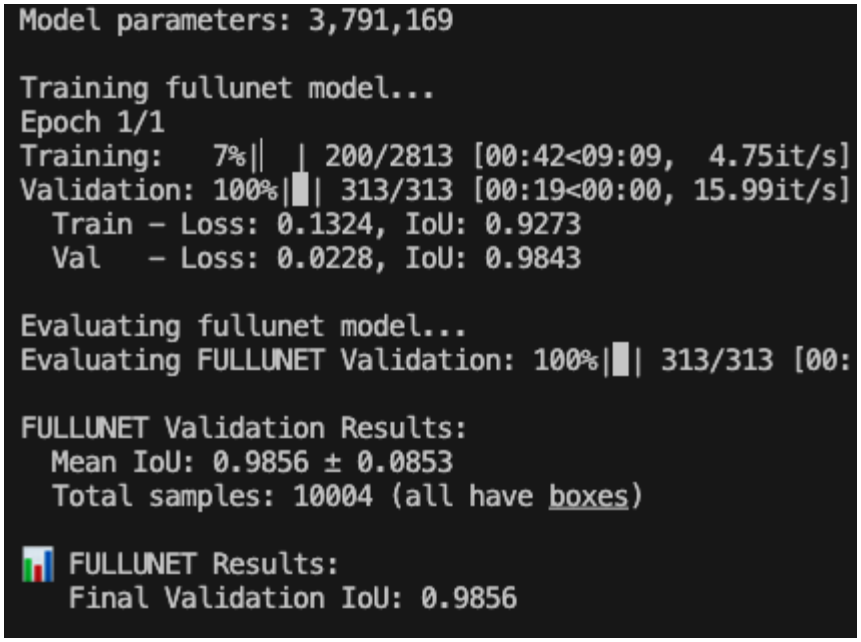
At our full optimization setting for our CNN classifier, the 3 plots above show that the validation accuracy tends to be higher than the training accuracy. However, at around the 35th epoch, the

difference between the validation and training accuracy plateaus, and we obtain our best validation accuracy at epoch 40.

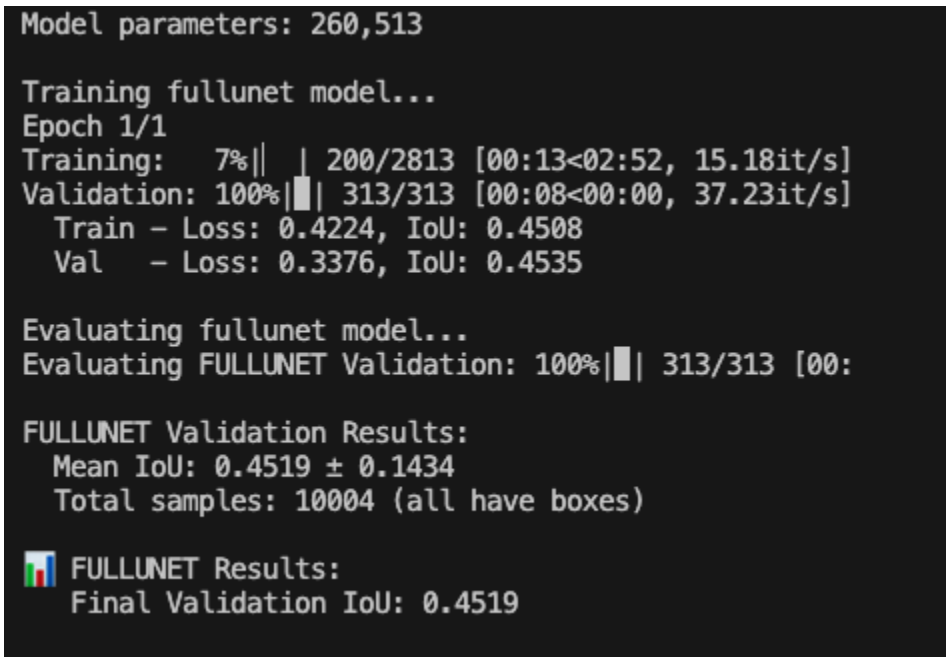




One particular parameter that influenced our UNet model's performance was the max_steps_per_epoch parameter. If we reduced it to 100, the final validation IoU score decreased. As a result, we found that a higher value for this parameter resulted in a higher validation score.



Also, when we reduced the depth of the model by removing the intermediate convolutional layers in both the encoder and decoder blocks, our model's performance decreased. Essentially, these were the convolutional layers that outputted the same number of output channels as the input, so having a model with a larger depth increased performance.



However, the most significant influence came from the width and kernel sizes. Reducing the channels by half across convolution layers, in addition to using kernel sizes ranging from 1-2,

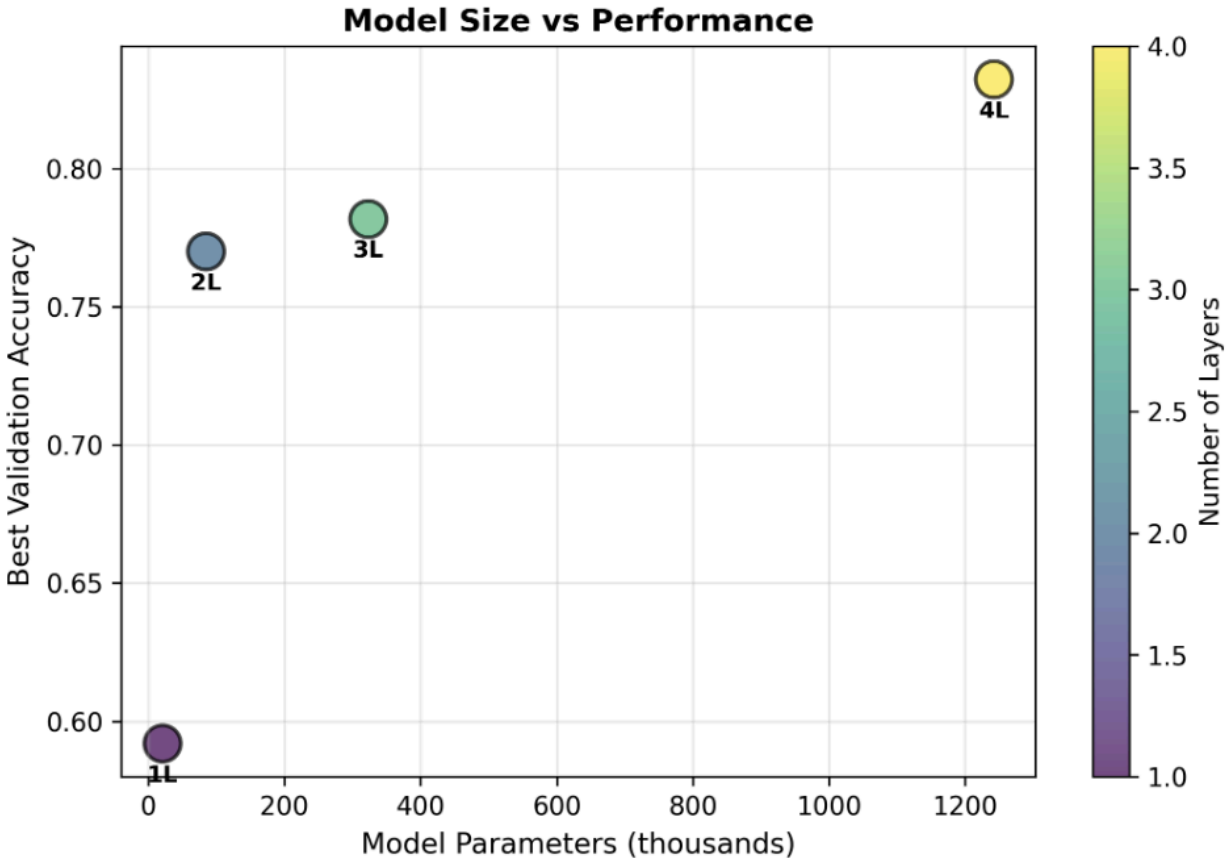
significantly reduced the performance of the UNet model. Having a larger kernel size, ideally 3, in addition having a wider model will extensively boost the performance.

An alternative and simpler CNN structure we used was 3 layers, which reaches a best validation accuracy of 77.9%. The structure is following:

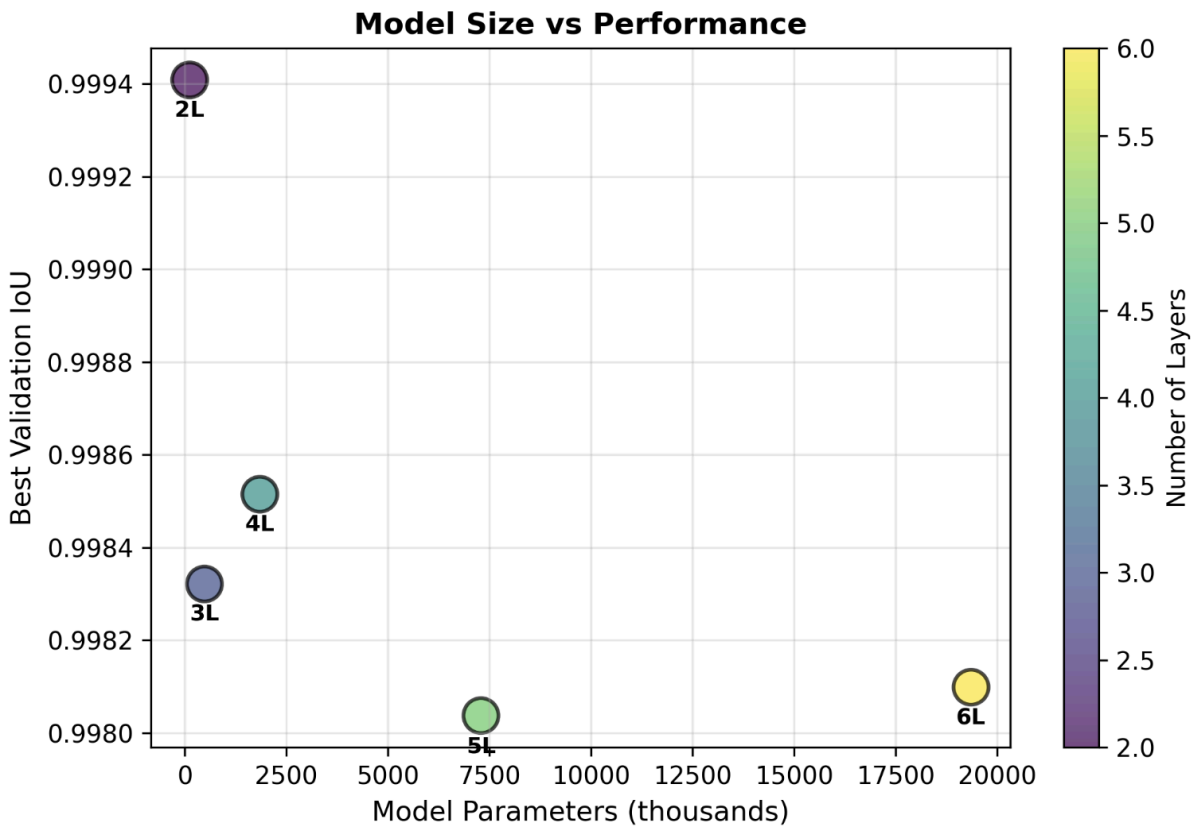
```
Python
(Conv2d(3, 32)
BatchNorm2d
ReLU
maxpool2d) # *3, Conv channels : (3 -> 32 -> 64 -> 128)

AdaptiveAvgPooling 128×3×3 → 128×1×1

Dropout(0.5)
Linear(128 → 256)
ReLU
Dropout(0.3)
Linear(256 → num_classes = 9)
```

The drop in performance prompted us to wonder how the number of layers will affect performance of CNN models for classification tasks. Indeed, the performance of CNN increases monotonically when the number of layers increases, but the growth rate significantly decreases after 2->3 layer. Moreover, deeper CNN needs more resources to be trained.

Layers vs Performance



We also investigated if the number of layers changes the performance of UNet for segmentation tasks. Interestingly, this relationship does not hold anymore, and performance actually decreases when the number of layers increases, though by a moderate amount. With N=2 layers, our UNet segmentation model already achieves validation IoU > 99%. We suspect that this is due to the fact that UNet models do not only rely on the number of convolutional layers but more importantly the bottleneck layer. If we fix the max number of channels, increasing the number of layers can shrink the dimension of the bottleneck layer (because of more pooling), which is not always ideal. Another possible reason is that our simulated segmentation task is simple enough that a complicated model isn't needed.



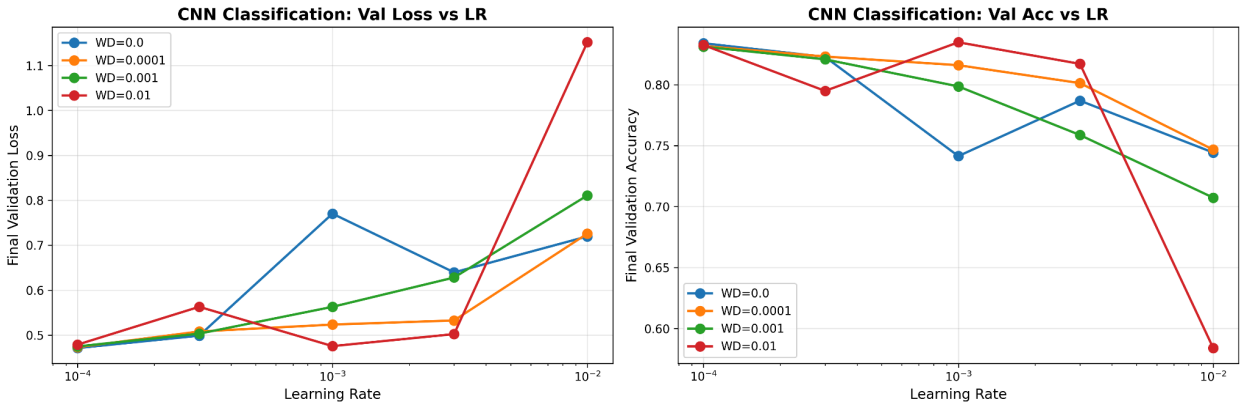
Hyperparameter Tuning

We performed a basic grid search on a ***lesser performing model*** with a different architecture for learning rate and weight decay (regularization) parameters with both the CNN used for classification and the UNET used for segmentation.

CNN Classification:

```
Python
(Conv2d(3, 32) # Conv2d output increased --> 64 --> 128
BatchNorm2d(32) # BatchNorm2d increased --> 64 --> 128
ReLU
maxpool2d)

Linear(128 → 256)
ReLU
```

```
Dropout(0.15)
Linear(256 → num_classes = 9)
```

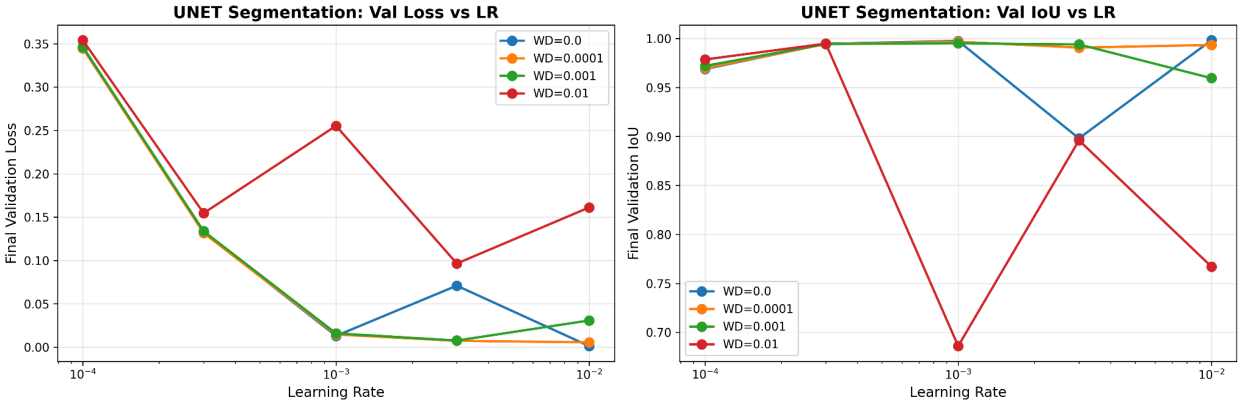


Notably, the model does not perform well using a weight decay (regularization of 0.01) even though it is the top performing model in the first six epochs.

UNET Segmentation:

```
Python
(Enc1: conv_block(in → B) → MaxPool2d(2))
(Enc2: conv_block(B → 2B) → MaxPool2d(2))

(Bottleneck: conv_block(2B → 4B))

(Up2: ConvTranspose2d(4B → 2B, k=2, s=2) → concat skip from Enc2 →
conv_block(4B → 2B))
(Up1: ConvTranspose2d(2B → B, k=2, s=2) → concat skip from Enc1 → conv_block(2B
→ B))

(Out: Conv2d(B → out_channels, k=1))
```

Best validation IoU score: 98.55%
Epochs = 1
Learning rate = 0.003
Weight decay = 0.0
Optimizer: Adam
Batch size = 32
Max steps per epoch = 200



Best U-Net segmentation - James

- Best accuracy IoU: 99.84%
- Mean accuracy IoU: 99.67%
- Learning rate: 0.001
- Weight decay: 0.0001
- Num epochs: 40
- Max steps per epoch: 300
- Batch size: 32
- Optimizer: Adam